LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE CR002
PERTEMUAN 3

Form Input dan State Management

Disusun Oleh:

Arum Syafitri Puspita Pratiwi (20240801010)

Dosen Pengampu:

JEFRY SUNUPURWA ASRI , S.Kom., M.Kom

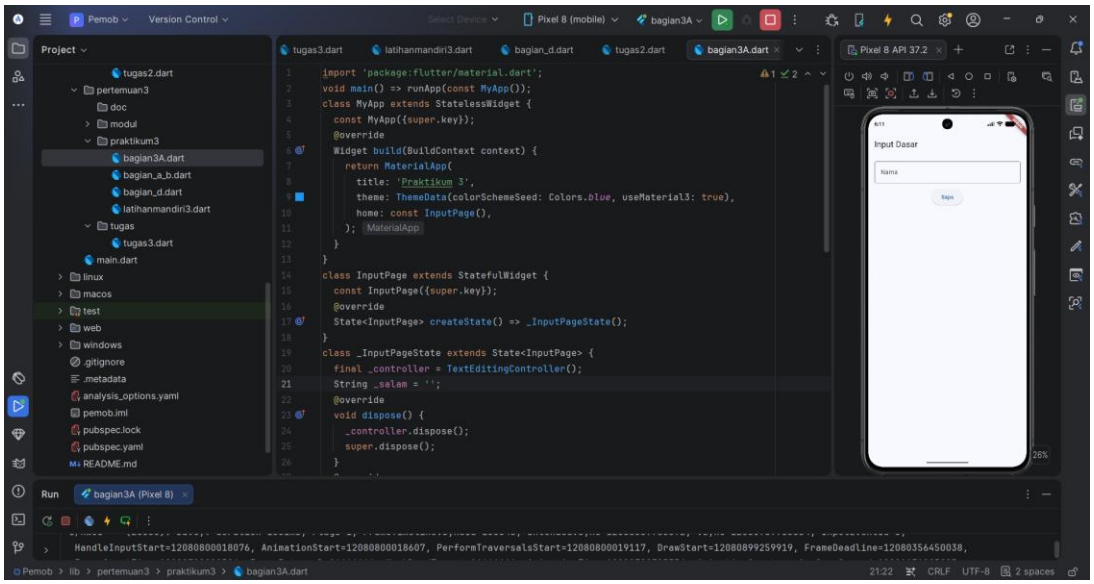
PROGRAM STUDI TEKNIK INFORMATIKA

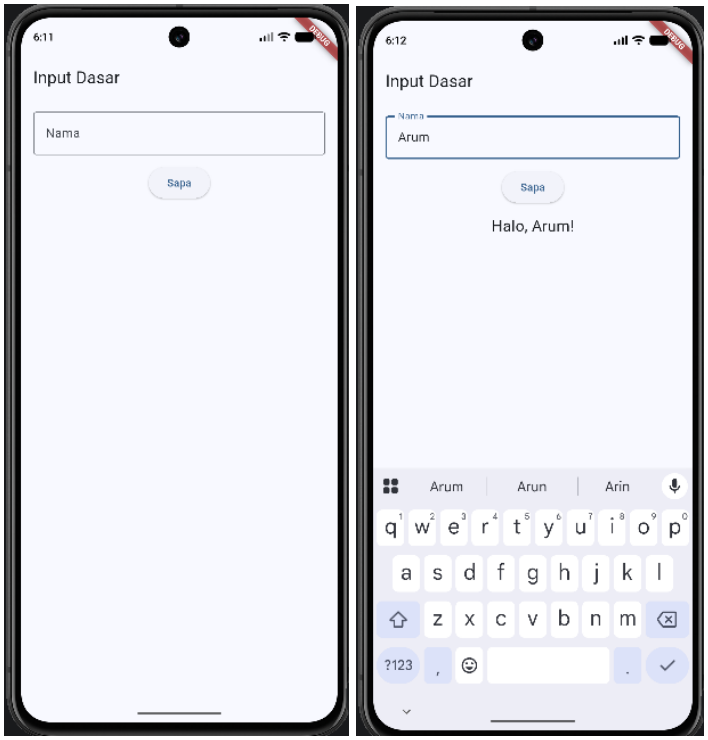
FAKULTAS ILMU KOMPUTER

UNIVERSITAS ESA UNGGUL

2026

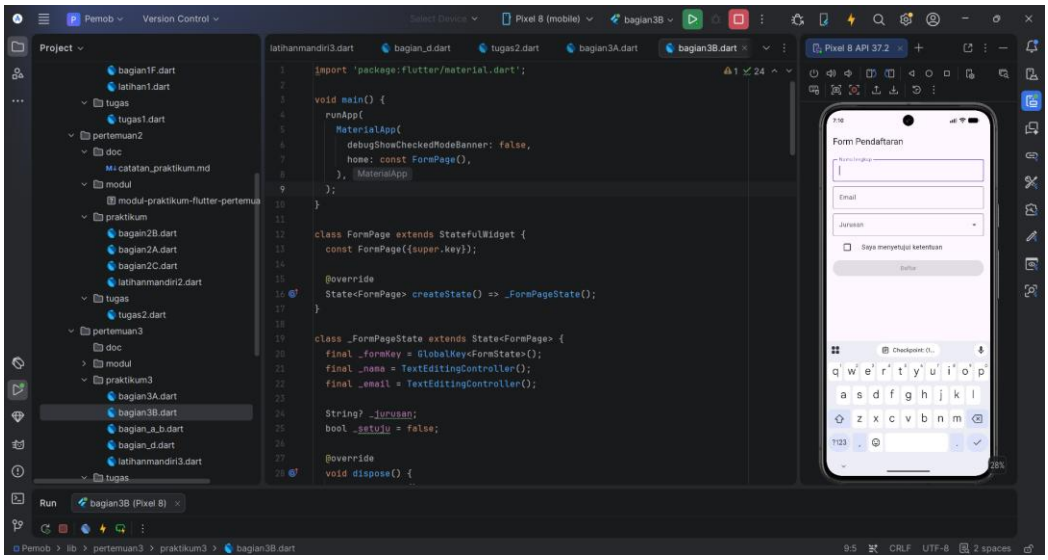
Bagian A: Input Dasar dengan TextField

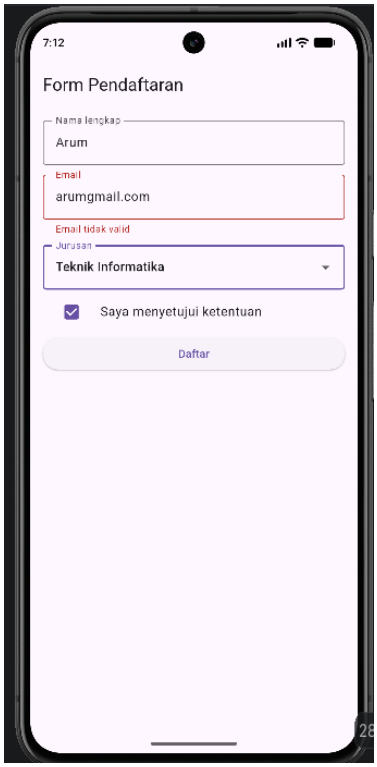


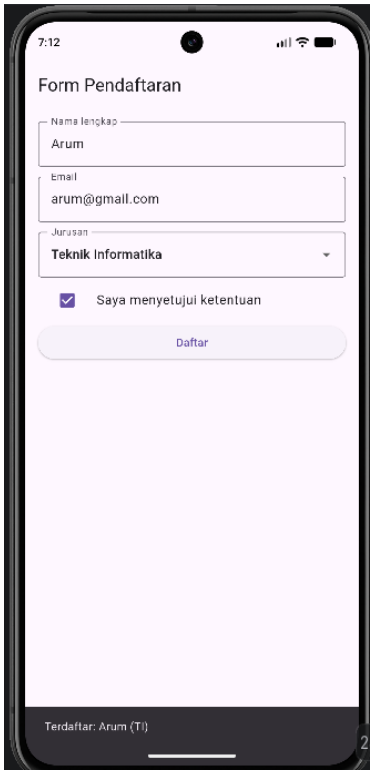


Pada bagian ini dibuat halaman input dasar menggunakan TextField untuk menerima masukan berupa nama dari pengguna. Data yang diketik pada TextField dikendalikan menggunakan TextEditingController, kemudian ketika tombol **Sapa** ditekan, nilai yang dimasukkan akan ditampilkan dalam bentuk pesan sapaan. setState() digunakan agar tampilan diperbarui setelah pengguna menekan tombol, sehingga muncul teks seperti “Halo, [nama]!” di bawah tombol.

Bagian B: Form dengan Validasi







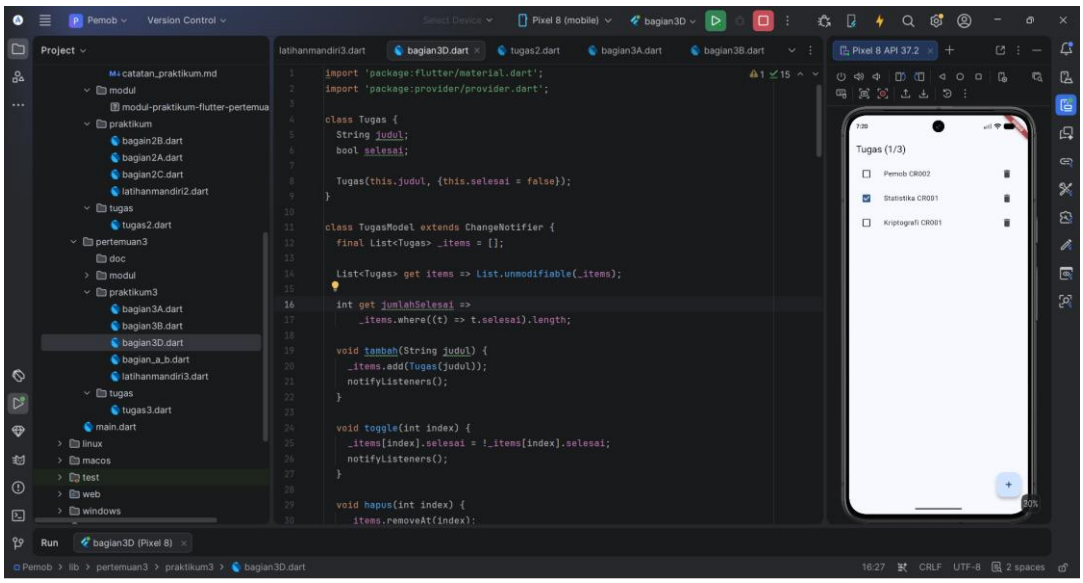
Pada bagian ini dibuat form pendaftaran yang terdiri dari input nama lengkap, email, pilihan jurusan, dan checkbox persetujuan. Setiap input memiliki validasi untuk memastikan data yang dimasukkan sesuai, seperti nama wajib diisi, email harus memiliki format yang valid, dan jurusan harus dipilih. Tombol **Daftar** hanya dapat digunakan setelah pengguna menyetujui ketentuan. Jika data berhasil divalidasi, aplikasi akan menampilkan SnackBar yang berisi informasi pendaftaran pengguna.

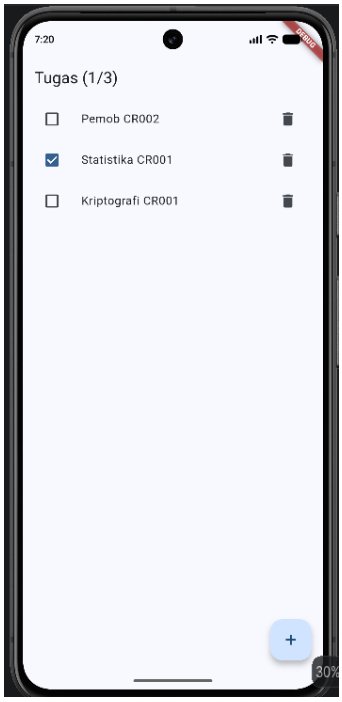
Bagian C: Mengapa Butuh State Management?

Pada bagian ini dijelaskan alasan penggunaan state management dalam aplikasi Flutter. Pada bagian sebelumnya, state masih dikelola menggunakan setState() di dalam satu halaman.

Namun, pada aplikasi seperti Daftar Tugas, halaman daftar dan halaman tambah membutuhkan data yang sama. Jika data harus dikirim melalui constructor dan callback antar halaman, pengelolaannya dapat menjadi lebih rumit. Oleh karena itu, data dapat dipindahkan ke satu objek bersama yang dapat diakses oleh beberapa halaman. Dalam Flutter, salah satu cara yang digunakan untuk mengelola data tersebut adalah dengan Provider.

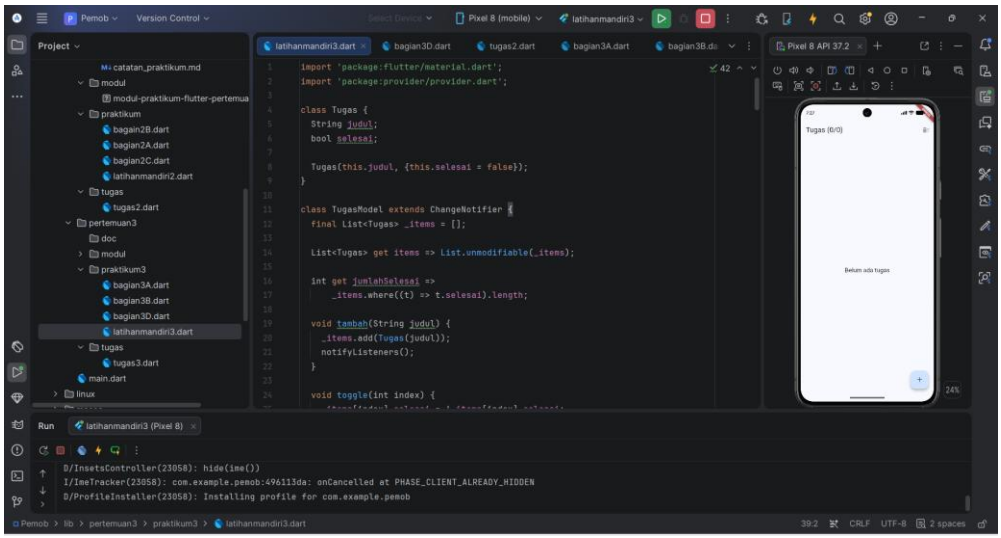
Bagian D: Aplikasi Daftar Tugas dengan Provider

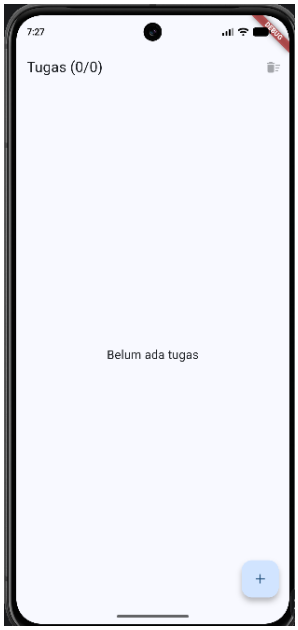


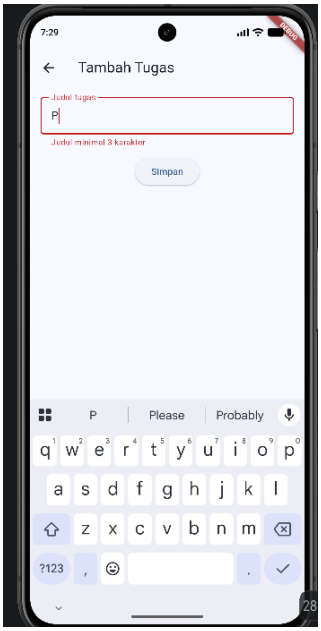


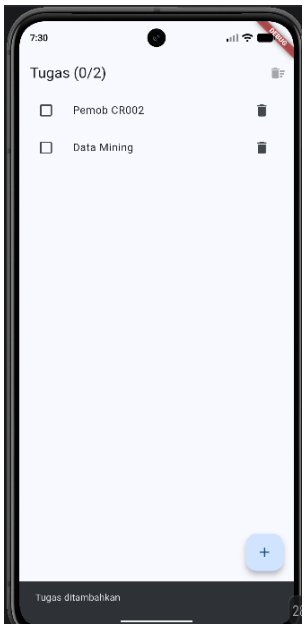
Pada bagian ini dibuat aplikasi Daftar Tugas menggunakan Provider untuk mengelola data tugas. Pengguna dapat menambahkan tugas, menandai tugas yang sudah selesai, dan menghapus tugas. Data dikelola melalui TugasModel dengan ChangeNotifier, sehingga perubahan data dapat langsung diperbarui pada tampilan aplikasi.

Latihan Mandiri





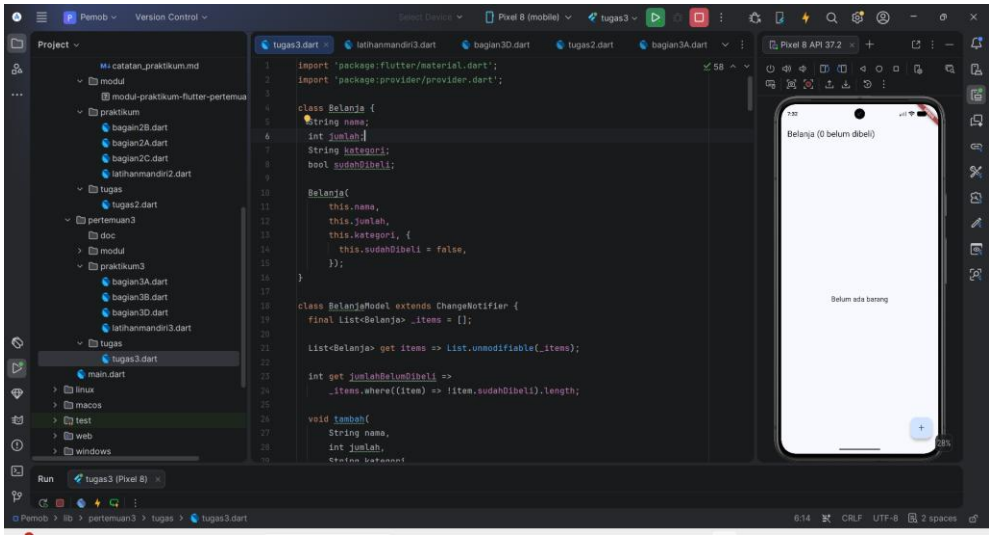


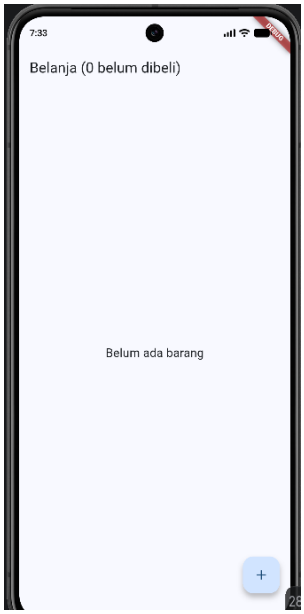


Pada latihan mandiri ini, aplikasi Daftar Tugas dikembangkan dengan menambahkan beberapa fitur. Pada halaman tambah tugas digunakan Form dan TextFormField untuk melakukan validasi judul minimal 3 karakter serta menampilkan pesan error jika input tidak memenuhi ketentuan. Ditambahkan juga method hapusSelesai() pada TugasModel yang dapat digunakan melalui tombol pada AppBar untuk menghapus seluruh tugas yang sudah selesai. Setelah tugas berhasil disimpan, aplikasi menampilkan SnackBar dengan pesan “Tugas ditambahkan”. Selain itu, ketika belum terdapat tugas, aplikasi akan menampilkan teks “Belum ada tugas” di tengah halaman.

Tugas

Buat aplikasi “Daftar Belanja”





Pada tugas ini dibuat aplikasi Daftar Belanja menggunakan Provider dan ChangeNotifier untuk mengelola state aplikasi. Halaman tambah barang menyediakan form yang terdiri dari nama barang, jumlah, dan kategori. Setiap isian memiliki validasi, yaitu nama barang wajib diisi, jumlah harus berupa angka lebih dari 0, dan kategori wajib dipilih. Setelah data berhasil ditambahkan, barang akan ditampilkan pada halaman daftar dengan informasi nama, jumlah, dan kategori. Pengguna dapat menandai barang yang sudah dibeli menggunakan checkbox atau menghapus barang dari daftar. Pada bagian AppBar ditampilkan jumlah barang yang belum dibeli sehingga jumlah tersebut dapat berubah sesuai dengan status barang.

Pertanyaan Refleksi

1. Mengapa TextEditingController harus di-dispose?

Jawab: Agar penggunaan memori dapat dibersihkan ketika halaman sudah tidak digunakan dan tidak menyebabkan penumpukan data.

2. Kapan cukup memakai setState, dan kapan sebaiknya beralih ke Provider?

   Jawab: setState cukup digunakan jika data hanya digunakan dalam satu halaman. Provider lebih cocok jika data perlu digunakan atau dibagikan oleh beberapa halaman.

3. Apa yang terjadi bila notifyListeners() lupa dipanggil? Mengapa?

   Jawab: Tampilan tidak langsung diperbarui meskipun datanya sudah berubah, karena widget yang menggunakan data tersebut tidak mendapat pemberitahuan bahwa terjadi perubahan.

4. Mengapa di dalam onPressed kita memakai context.read, bukan context.watch?

   Jawab: Karena onPressed hanya membutuhkan akses untuk menjalankan fungsi atau mengubah data, bukan untuk memantau perubahan data secara langsung.

Kesimpulan

Pada pertemuan ini dipelajari cara menerima input pengguna, membuat form dengan validasi, serta mengelola state pada aplikasi Flutter. Materi diterapkan melalui penggunaan TextField, TextEditingController, Form, dan TextFormField. Selain itu, dipelajari penggunaan ChangeNotifier dan Provider untuk mengelola data yang digunakan oleh beberapa halaman, termasuk penggunaan context.watch dan context.read. Materi kemudian diterapkan dalam pembuatan aplikasi Daftar Tugas dan Daftar Belanja dengan fitur penambahan, pengubahan status, penghapusan data, validasi, dan pengelolaan state.